

Módulo 8 – Capítulo 04 – Sección 01

GPU vs CPU vs Apple Silicon: cuándo usar cada arquitectura

Las herramientas presentadas en el capítulo anterior —llama.cpp con sus flags de aceleración y Ollama con su detección automática de hardware— asumen que el hardware correcto ya está disponible. Pero ¿cuál es el hardware correcto? La respuesta depende del tamaño del modelo, la cuantización elegida, el throughput requerido y el presupuesto disponible, y varía significativamente entre los tres tipos de arquitectura de procesamiento disponibles en 2025: GPUs NVIDIA con memoria dedicada, CPUs modernas con instrucciones SIMD, y los chips Apple Silicon con su arquitectura de memoria unificada.

Las GPUs NVIDIA son el estándar de producción para modelos de 7B parámetros o más que requieren alta velocidad de generación. Su arquitectura SIMD con miles de núcleos CUDA opera las multiplicaciones matriciales de los transformers con un throughput 10-50x superior a una CPU de última generación en operaciones de inferencia de FP16 o INT8. La restricción fundamental es que el modelo completo debe caber en VRAM para máxima eficiencia: si el modelo en la cuantización elegida no cabe en la VRAM disponible, el rendimiento cae drásticamente debido al offloading a RAM del sistema a través del bus PCIe, que tiene una fracción del ancho de banda de la VRAM HBM. Una GPU de 24 GB de VRAM (como la RTX 4090 o la RTX 3090) es suficiente para modelos de hasta 13B en Q4_K_M o hasta 7B en BF16 completo.

La inferencia en CPU es el territorio de los modelos cuantizados de 3B-7B cuando la GPU no está disponible o cuando la latencia absoluta (tiempo desde el envío hasta recibir la respuesta completa) es aceptable aunque el tiempo por token sea mayor. llama.cpp con instrucciones AVX2 en un procesador moderno de 16 núcleos (como un AMD Ryzen 9 o Intel Core i9) puede generar 15-25 tokens/s con un modelo Q4_K_M de 7B, usando directamente la RAM del sistema como "memoria del modelo". Este rendimiento es suficiente para muchas aplicaciones donde el usuario espera una respuesta de varios segundos: clasificación de documentos en batch, generación de resúmenes, extracción de información no interactiva. La ventaja de la CPU es el costo: cualquier servidor moderno con 32 GB de RAM puede ejecutar modelos de 7B sin GPU dedicada.

Apple Silicon (chips M1, M2, M3, M4) ocupa un nicho único y extraordinariamente eficiente en el espectro de hardware para LLMs. La arquitectura de memoria unificada (UMA) de estos chips elimina el cuello de botella de transferencia PCIe que limita el offloading en sistemas x86+GPU discreta: la CPU, la GPU integrada y el Neural Engine comparten el mismo pool físico de RAM LPDDR5, de modo que toda la RAM del sistema actúa como VRAM efectiva. En una MacBook Pro con M3 Max y 96 GB de memoria unificada, es posible cargar Llama 3 70B en Q4_K_M —un archivo de ~40 GB— completamente en "VRAM" y generar tokens a 15-25 tokens/s, un rendimiento que ninguna configuración de GPU discreta de 24 GB podría alcanzar para ese tamaño de modelo sin particionado multi-GPU. Para los modelos de 7B-13B en variantes Q4 o Q8, un M2 Pro con 32 GB ofrece velocidades de 30-60 tokens/s con Flash Attention via MLX, superando frecuentemente a llama.cpp+CUDA en GPUs de consumo de precio similar.

Cuándo usar cada arquitectura

- **GPU NVIDIA dedicada:** óptima para producción con múltiples usuarios concurrentes; necesaria para fine-tuning con QLoRA; modelos de 7B+ con Q4 en GPUs de 8-12 GB VRAM; modelos de 70B requieren 48+ GB VRAM o multi-GPU.
- **CPU moderna (x86-64 con AVX2):** adecuada para desarrollo local, inferencia en batch o cuando el presupuesto no permite GPU dedicada; modelos de hasta 13B en Q4_K_M con 32 GB RAM; 5-20 tokens/s aceptable para tareas no interactivas.
- **Apple Silicon (M-series):** el punto óptimo para desarrollo y producción de baja-media escala; un M3 Max con 128 GB puede ejecutar Llama 3 70B en Q4 a 15-25 tokens/s; la UMA elimina el límite de VRAM de las GPUs discretas.
- **GPU AMD con ROCm:** viable en Linux con soporte ROCm 6.x; soporte mejorado en llama.cpp y vLLM pero más inestable que CUDA; GPUs Radeon RX 7900 XTX (24 GB VRAM) competitivas en precio/rendimiento para 7B-13B.
- **Aceleradores especializados:** Intel Gaudi y Google TPUs para fine-tuning a escala; NPUs en dispositivos móviles para modelos 1B-3B en el borde; relevantes para casos de uso muy específicos.

Nota del Arquitecto: La pregunta que siempre hago cuando un equipo me dice que "no puede permitirse GPU" es: ¿tienen laptops Mac con M-series? Un M2 Pro con 16 GB de memoria unificada corre modelos de 7B a 30-40 tokens/s, suficiente para la mayoría del trabajo de desarrollo. Si el equipo tiene laptops Windows con GPU integrada Intel o AMD, la CPU es la opción: lenta pero funcional para evaluación.

Conocer las características de cada arquitectura de hardware permite tomar decisiones de infraestructura fundamentadas en los requisitos reales del modelo y el caso de uso. La sección siguiente proporciona las fórmulas para calcular exactamente cuánta VRAM (o RAM) necesita cualquier modelo en cualquier cuantización, convirtiendo este análisis cualitativo en planificación cuantitativa.

Módulo 8 – Capítulo 04 – Sección 02

VRAM como restricción principal: calcular los requisitos de memoria por modelo y cuantización

El Capítulo 2 estableció la fórmula canónica de VRAM para los pesos del modelo: `parámetros × bits_cuantización / 8 / 1e9`. Esta fórmula es el punto de partida del análisis de memoria, pero es insuficiente para planificar un despliegue real porque omite el componente que frecuentemente causa los OOM errors en producción: el KV cache. Los pesos del modelo son estáticos —se cargan una vez y permanecen en memoria mientras el servidor está activo— pero el KV cache crece con cada petición activa, se multiplica con el número de peticiones concurrentes y escala con la longitud del contexto. Un equipo que planifica la VRAM solo considerando los pesos puede encontrar que su modelo de 7B en Q4_K_M, que ocupa 3.5 GB de pesos, necesita 6-8 GB de VRAM total cuando se incluye el KV cache para servir cuatro peticiones simultáneas con contexto de 4096 tokens.

La fórmula de VRAM para el KV cache de una petición activa es:

```
KV_bytes = 2 × n_layers × n_heads × head_dim × context_tokens × bytes_por_elemento
```

Para Llama 3 8B con sus 32 capas, 32 cabezas de atención, dimensión de cabeza de 128 y contexto de 8192 tokens en FP16 (2 bytes por elemento):

```
KV_bytes = 2 × 32 × 32 × 128 × 8192 × 2 = 4.3 GB
```

Este cálculo explica por qué incrementar el contexto máximo tiene un costo de VRAM lineal: pasar de 4096 a 8192 tokens duplica el KV cache de esa petición. En vLLM con `--max-model-len 8192`, el engine reserva VRAM para el KV cache pool asumiendo que el sistema puede estar sirviendo hasta N peticiones simultáneas con contextos de esa longitud máxima. Si se especifica `--gpu-memory-utilization 0.9` en un servidor con 24 GB de VRAM, con el modelo de 7B Q4 ocupando 3.5 GB, quedan ~18 GB disponibles para el KV cache pool —suficiente para aproximadamente 4-5 peticiones simultáneas con contexto de 8192 tokens en FP16.

Los modelos con GQA (Grouped Query Attention), como Llama 3, reducen el tamaño del KV cache significativamente respecto a la atención multi-cabeza estándar. En Llama 3 8B, el número de cabezas de atención de las claves y valores (KV heads) es 8 en lugar de las 32 cabezas de query, lo que reduce el KV cache a un cuarto del tamaño que tendría con atención multi-cabeza completa. Esta es una de las razones por las que los modelos con GQA son más eficientes en serving concurrente: el mismo hardware puede servir más peticiones simultáneas con el mismo presupuesto de VRAM.

El overhead del framework es el tercer componente de memoria, frecuentemente fijo entre 500 MB y 1.5 GB dependiendo del runtime: el contexto CUDA reserva ~500 MB en GPUs NVIDIA, el PyTorch memory allocator retiene bloques liberados para reutilización futura, y vLLM mantiene buffers internos para el scheduler y el continuous batching. La regla práctica es añadir un 10-15% al total de pesos + KV cache para este overhead, verificando siempre con `nvidia-smi` tras el inicio del servidor que los números coinciden con la planificación.

Cálculo de requisitos de VRAM

- **Fórmula de pesos** (de Cap. 2): `VRAM_pesos = parámetros × bits_cuantización / 8 / 1e9`; para 7B Q4: 3.5 GB; Q8: 7 GB; BF16: 14 GB.
- **KV cache por petición**: `2 × n_layers × n_heads_kv × head_dim × context_len × bytes_por_elemento`; para Llama 3 8B con 8K tokens en FP16: ~4.3 GB; con GQA (8 KV heads): ~1.1 GB.
- **Overhead del framework**: 10-15% adicional sobre pesos + KV cache; ~500 MB a 1.5 GB fijos para el runtime CUDA/Metal.
- **VRAM para fine-tuning con QLoRA**: modelo base en NF4 (mitad que BF16) + gradientes de adaptadores LoRA + optimizador AdamW 8-bit = ~8 GB mínimo para fine-tuning de 7B.
- **Multi-GPU con tensor parallelism**: vLLM con `--tensor-parallel-size=N` divide los pesos entre N GPUs; la VRAM disponible es la suma de todas, con overhead de comunicación que reduce el throughput efectivo ~10-15%.

Nota del Arquitecto: El error de planificación de VRAM más común que he visto es ignorar el KV cache y lanzar el servidor solo para obtener un OOM error al recibir las primeras peticiones largas. El proceso correcto es: (1) calcular `VRAM_pesos` con la fórmula del Cap. 2; (2) calcular el KV cache máximo para el número de peticiones concurrentes esperadas con el contexto máximo del producto; (3) sumar el overhead

del 12%; (4) comparar con la VRAM disponible; (5) si no cabe, reducir la longitud máxima de contexto o el número de peticiones concurrentes antes de cambiar el hardware.

Con la fórmula completa de VRAM en mano, el AI Engineer puede planificar el hardware de cualquier despliegue con precisión. La sección siguiente traduce este análisis cuantitativo al catálogo de GPUs disponibles en el mercado en 2025, con sus características técnicas concretas.

Módulo 8 – Capítulo 04 – Sección 03

GPUs para inferencia: NVIDIA RTX/A-series, AMD Instinct y sus características

El análisis cuantitativo de VRAM del apartado anterior necesita mapearse a hardware concreto disponible en el mercado. El ecosistema de GPUs para inferencia de LLMs está segmentado en tres categorías con perfiles técnicos muy distintos: GPUs de consumo (RTX series), GPUs profesionales de estación de trabajo (A-series NVIDIA, Radeon Pro) y aceleradores de datacenter (H100/A100 NVIDIA, MI300X AMD). Cada categoría tiene diferentes capacidades de VRAM, ancho de banda de memoria, soporte de formatos de precisión y ecosistemas de software, y la selección correcta depende del tamaño del modelo, el throughput requerido y el entorno de despliegue.

Las GPUs de la familia RTX 4000/5000 de NVIDIA son el punto de entrada más accesible para inferencia local de LLMs en hardware de consumo. La RTX 4090 con 24 GB de GDDR6X y 1.008 TB/s de ancho de banda de memoria es la GPU de consumo más potente disponible: puede ejecutar modelos de hasta 13B en Q8 (13 GB de pesos) o hasta 34B en Q4 (17 GB de pesos) dentro de su VRAM, con velocidades de generación de 40-80 tokens/s para modelos de 7B en Q4. El diseño con Tensor Cores Ada Lovelace soporta operaciones INT8 e INT4 aceleradas, además del nuevo tipo de dato FP8 en las variantes RTX 40-series, aunque el soporte de software para FP8 en inferencia de LLMs es más maduro en las GPUs de datacenter H100. Las GPUs de 16 GB de la misma generación (RTX 4080 Super, RTX 4070 Ti Super) son suficientes para modelos de 7B en Q8 o hasta 13B en Q4_K_M con contextos moderados.

Las GPUs de datacenter NVIDIA A100 y H100 son el estándar de producción para serving de LLMs a escala. El A100 en sus variantes de 40 GB y 80 GB usa memoria HBM2e con anchos de banda de 1.55 TB/s y 2 TB/s respectivamente, muy superior al GDDR6X de las RTX: este ancho de banda es el factor determinante en el rendimiento de decode, que es memory-bound. Múltiples A100 conectados por NVLink forman configuraciones multi-GPU donde vLLM puede distribuir un modelo de 70B entre dos o cuatro GPUs usando tensor parallelism. El H100 en 80 GB con HBM3 a 3.35 TB/s añade soporte nativo para FP8 con el Transformer

Engine de NVIDIA, que puede lograr throughput 2-3x superior a BF16 con degradación de calidad mínima cuando se combina con TensorRT-LLM —técnica explorada en profundidad en el Capítulo 5.

AMD ha reposicionado sus GPUs de datacenter Instinct MI300X como una alternativa directa a las GPUs NVIDIA para inferencia de LLMs. El MI300X ofrece 192 GB de HBM3 —más que cualquier GPU NVIDIA disponible— con 5.3 TB/s de ancho de banda de memoria, lo que lo hace particularmente eficiente para modelos muy grandes (70B-405B) que requieren capacidad de VRAM pero donde el número de peticiones concurrentes es moderado. El ecosistema de software ROCm de AMD ha madurado significativamente: vLLM, PyTorch y llama.cpp tienen soporte ROCm funcional, aunque la profundidad de optimización de kernels es menor que en CUDA. Lambda Labs y Azure ofrecen instancias con MI300X para organizaciones que buscan alternativas a las GPUs NVIDIA en la nube.

Al evaluar GPUs, el criterio más relevante no es el precio absoluto sino la relación entre capacidad de VRAM, ancho de banda de memoria y el rango de precios actual del mercado. Los precios de hardware para inferencia varían con la disponibilidad y la demanda; se recomienda consultar los precios actuales en los sitios oficiales de los fabricantes y comparadores especializados antes de tomar decisiones de compra.

GPUs relevantes para inferencia de LLMs

- **NVIDIA RTX 4090:** 24 GB GDDR6X, 1.008 TB/s de ancho de banda, 82.6 TFLOPS FP16; soporte INT8 e FP8 via Tensor Cores Ada; para modelos de 7B-34B con Q4-Q8; punto de entrada más eficiente para inferencia local de producción.
- **NVIDIA RTX 3090/4080 (16 GB):** para modelos de 7B en BF16 completo o 13B en Q4; soporte INT8 en Tensor Cores; opción más económica para desarrollo y producción de baja escala.
- **NVIDIA A100 (40/80 GB HBM2e):** ancho de banda 1.55/2 TB/s; soporte NVLink multi-GPU; estándar para serving de 13B-70B a escala; disponible principalmente en cloud (AWS p4d, GCP a2, Lambda Labs).
- **NVIDIA H100 (80 GB HBM3):** 3.35 TB/s ancho de banda, soporte FP8 nativo con Transformer Engine; throughput 3x superior a A100 en inferencia de LLMs; para producción de alta demanda donde el costo por token es crítico.
- **AMD Instinct MI300X (192 GB HBM3):** mayor VRAM del mercado; soporte ROCm con vLLM y PyTorch; throughput competitivo con H100 para modelos grandes; disponible en Lambda Labs y Azure.

Nota del Arquitecto: Para la mayoría de los equipos que comienzan con inferencia local de LLMs en producción, el punto de inicio más eficiente es una GPU de consumo de 24 GB de la gama más reciente disponible en el momento de la compra. Con esa VRAM, un modelo de 13B en Q4_K_M tiene margen suficiente para contextos de hasta 4096 tokens con 4-6 peticiones simultáneas. Cuando el throughput requerido supera ese límite, la escala horizontal (múltiples instancias en la nube) es generalmente más económica que escalar verticalmente a GPUs de datacenter para la mayoría de los patrones de uso.

Las características técnicas de las GPUs determinan qué modelos y qué cuantizaciones son viables en cada punto de precio. La sección siguiente explora la ventaja específica que ofrece Apple Silicon con su arquitectura de memoria unificada para un subconjunto importante de casos de uso de inferencia local.

Módulo 8 – Capítulo 04 – Sección 04

Apple Silicon: MLX y la ventaja de la memoria unificada para modelos medianos

Las GPUs de consumo de 24 GB representan actualmente el techo de VRAM para hardware local con arquitectura discreta GPU+CPU. Más allá de ese límite, la única opción sin recurrir a multi-GPU o hardware de datacenter es Apple Silicon: los chips M-series con su arquitectura de memoria unificada ofrecen hasta 192 GB de memoria compartida entre CPU y GPU en los modelos Mac Pro y Mac Studio con chips M2 Ultra/M4 Ultra, con características técnicas que los hacen particularmente apropiados para inferencia de modelos medianos (13B-70B) que no caben en GPUs de consumo discretas.

La ventaja central de Apple Silicon para inferencia de LLMs es la eliminación del cuello de botella PCIe. En una arquitectura tradicional x86 + GPU discreta, cuando un modelo no cabe completamente en VRAM y se recurre al offloading de capas a RAM del sistema (como permite el flag `--n-gpu-layers` de llama.cpp), los datos deben transferirse a través del bus PCIe con un ancho de banda de 32-64 GB/s en PCIe 4.0. Este ancho de banda es entre 10 y 100 veces menor que el ancho de banda de VRAM, produciendo un cuello de botella severo que puede reducir el rendimiento del decode a 1-5 tokens/s para modelos grandes. En Apple Silicon, la GPU integrada y la CPU acceden al mismo pool físico de memoria LPDDR5 con anchos de banda de 200-800 GB/s según el chip y generación, eliminando completamente esta penalización de transferencia.

El resultado práctico es que en un Mac Studio con M3 Ultra y 192 GB de memoria unificada, un modelo Llama 3 70B en Q4_K_M (~40 GB) carga completamente en "VRAM efectiva" y genera tokens a 15-25 tokens/s —rendimiento comparable a una sola A100 de 40 GB en GPU de datacenter. Para modelos de 7B-13B, los chips M-series de gama media ofrecen velocidades de 40-80 tokens/s con MLX, superando frecuentemente a GPUs de consumo de precio similar por la eficiencia del ancho de banda unificado y la optimización específica de la arquitectura.

MLX (Machine Learning eXplore) es el framework de aprendizaje automático nativo de Apple Research, diseñado desde cero para la arquitectura de memoria unificada. Su modelo de ejecución lazy y sus operaciones optimizadas para el chip permiten eficiencias que llama.cpp con el backend Metal no siempre alcanza, especialmente en modelos de 7B-34B. La librería `mlx-lm` (instalable con `pip install mlx-lm`) implementa inferencia para todos los modelos principales con cuantización nativa en 4 bits: `mlx_lm.generate(model, tokenizer, prompt="...")` ejecuta generación directamente en Apple Silicon sin dependencias adicionales. La conversión de modelos de Hugging Face al formato MLX se realiza con `mlx_lm.convert --hf-path <modelo> --mlx-path <destino> -q --q-bits 4`, un proceso que tarda 5-15 minutos dependiendo del tamaño del modelo.

MLX también soporta fine-tuning con LoRA directamente en Apple Silicon: `mlx_lm.lora --model <modelo> --data <datos> --iters 1000 --lora-layers 8` entrena adaptadores LoRA en el mismo hardware de inferencia. Para modelos de 7B en un M2 Pro con 32 GB de memoria, este proceso tarda 2-4 horas para un dataset de 1.000-5.000 ejemplos, con velocidad comparable a QLoRA en una GPU de 24 GB. Esta capacidad convierte un Mac de gama profesional en un entorno de desarrollo completo para evaluación, fine-tuning e inferencia de LLMs locales sin depender de hardware de datacenter ni de cloud GPU.

Aspectos técnicos de Apple Silicon para LLMs

- **Ancho de banda de memoria:** M3 Max y Ultra ofrecen 400-800 GB/s de ancho de banda unificado; comparable o superior a algunas GPUs de datacenter; factor determinante en velocidad de generación (memory-bound).
- **MLX vs llama.cpp en Apple Silicon:** MLX supera a llama.cpp+Metal en tokens/s para modelos 7B-34B por estar diseñado desde cero para UMA; llama.cpp tiene más variedad de formatos y mayor portabilidad.
- **Cuantización nativa en MLX:** `mlx_lm.convert` convierte modelos Hugging Face a INT4 nativo de MLX; los archivos resultantes son npz/safetensors con metadatos de cuantización propios del formato MLX.
- **Fine-tuning LoRA en MLX:** viable para modelos de hasta 7B en M2 Pro/Max con 32-64 GB; velocidad comparable a QLoRA en GPU de 24 GB para datasets pequeños y medianos.
- **Limitaciones:** sin soporte multi-GPU (los chips Ultra usan die-to-die interconnect, no GPUs independientes); vLLM y TensorRT no soportan Apple Silicon para producción escalable; ecosistema de software más reducido que CUDA.

Nota del Arquitecto: *El caso de uso donde Apple Silicon brilla sin competencia es el desarrollo y evaluación de modelos de 13B-70B sin acceso a hardware de datacenter. Un Mac Studio M3 Ultra con 192 GB puede servir como entorno de desarrollo completo para cualquier proyecto con modelos open weights: evaluación con el golden dataset, fine-tuning LoRA en datasets de miles de ejemplos, e inferencia en producción de baja escala. Para producción a escala alta, el salto a GPU de datacenter en la nube es inevitable.*

Apple Silicon con MLX amplía el espacio de hardware viable para inferencia de modelos medianos y grandes, complementando el ecosistema de GPUs NVIDIA. La sección siguiente cierra el análisis de hardware con la comparación de costo-rendimiento que permite tomar la decisión de compra o alquiler con los números correctos.

Módulo 8 – Capítulo 04 – Sección 05

Comparación costo-rendimiento: GPU en la nube vs hardware dedicado local

La decisión entre GPU en la nube y hardware local es, en su esencia, un análisis de punto de equilibrio: cuántas horas de uso se necesitan para que el costo acumulado de alquilar supere el costo de comprar. Sin embargo, los equipos frecuentemente cometen el error de comparar solo el precio del hardware con el precio de las instancias en la nube, ignorando los costos operativos del hardware propio y sobrestimando las horas de uso real. Un análisis riguroso incluye todos los componentes de costo en ambas opciones.

Para el hardware local, los costos relevantes son: el costo de capital del hardware (GPU, sistema, almacenamiento), amortizado sobre la vida útil estimada del equipo (típicamente 3-4 años para GPU de consumo, 4-6 para GPU profesional); la electricidad (una GPU de consumo de alto rendimiento consume 300-450W bajo carga, lo que a tarifas europeas representa un costo anual considerable para uso intensivo continuo); el cooling y el espacio (en una oficina con HVAC estándar, las GPUs de alto rendimiento pueden requerir cooling adicional que tiene costo real); y el tiempo de ingeniero para gestionar el hardware (actualizaciones de drivers, reemplazos de componentes, troubleshooting). El costo de capital dividido entre los meses de vida útil da el costo mensual amortizado; sumado al costo operativo mensual y dividido entre las peticiones o tokens generados en ese mes, produce el costo por token del hardware local.

Para la GPU en la nube, el análisis es más directo en apariencia pero requiere distinguir entre distintos modelos de pricing: las instancias on-demand de los hiperescaladores (AWS, GCP, Azure) tienen el precio más alto pero el mayor nivel de servicio y disponibilidad garantizada; los proveedores especializados (Lambda Labs, RunPod, Vast.ai) ofrecen instancias similares a precios significativamente más bajos; las instancias spot o preemptible de todos los proveedores ofrecen descuentos del 60-90% a cambio de poder ser interrumpidas con aviso corto. Los precios cambian frecuentemente y varían por región y tipo de GPU —consultar siempre los precios actualizados en los sitios oficiales de los proveedores antes de cualquier decisión de arquitectura.

El análisis de break-even para una GPU de consumo de 24 GB de VRAM del segmento premium frente a una instancia cloud equivalente con 24 GB depende críticamente del patrón de uso: si el equipo usa la GPU 8 horas al día durante días laborables, el punto de equilibrio con la instancia cloud equivalente en on-demand puede alcanzarse en 3-6 meses, haciendo que el hardware local sea más económico a largo plazo. Si el uso es menos de 4 horas diarias o muy intermitente, la nube es más económica. Para GPUs de datacenter (A100, H100), el costo de capital por unidad es muy superior y el break-even con proveedores especializados puede tomar 12-24 meses, lo que hace que la nube sea preferible para la mayoría de los equipos que no tienen uso continuo al 70-80%.

Las instancias de proveedores especializados como Lambda Labs ofrecen instancias A100 a precios significativamente inferiores a los hiperescaladores en sus tarifas on-demand, aunque con menor ecosistema de servicios integrados y menores garantías de SLA. Para workloads de entrenamiento que toleran interrupciones, las instancias Spot de los hiperescaladores ofrecen 60-70% de descuento respecto a on-demand y son la opción estándar para fine-tuning con checkpoint frecuente.

Comparación costo-rendimiento

- **Hardware local de consumo (24 GB VRAM):** costo de capital alto pero amortizable en 12-24 meses con uso de 8+ horas diarias; costo operativo incluye electricidad, cooling y tiempo de ingeniero; máximo control, latencia de red cero, sin costos de egress.
- **GPU en nube on-demand (hiperescaladores):** precio más alto pero disponibilidad inmediata y SLA garantizado; ideal para workloads de entrenamiento episódicos o experimentos; sin costo de capital inicial.
- **GPU en nube (proveedores especializados como Lambda Labs, RunPod):** precios intermedios entre on-demand de hiperescaladores y hardware local; sin el ecosistema de servicios de los hiperescaladores; soporte para GPUs de alto segmento a precios accesibles.
- **Instancias Spot/Preemptible:** 60-90% de descuento respecto on-demand; adecuadas para entrenamiento con checkpoint frecuente; no recomendadas para inferencia de producción con SLA.
- **Break-even real:** calcular con el volumen real de uso (horas/día, días/mes), el precio del hardware amortizado a 3-4 años y el precio de la instancia equivalente; el cloud es generalmente más económico por debajo de 4 horas de uso diario; el hardware propio es más económico por encima de 8 horas.

Nota del Arquitecto: *El error más frecuente en este análisis es olvidar el costo de electricidad y el tiempo de ingeniero para gestionar el hardware local. Una GPU que funciona bien durante dos años sin problemas tiene un costo oculto de cero, pero ese escenario no siempre se cumple: los fallos de hardware requieren tiempo de diagnóstico, garantía/reemplazo y downtime. Incluye un 20% de overhead sobre el costo de capital para cubrir estos costos operativos reales antes de comparar con la nube.*

El análisis de costo-rendimiento produce un número concreto: el costo por millón de tokens generados para cada opción con el volumen de uso esperado. Este número es la base para la decisión de arquitectura de hardware. El capítulo cierra con el principio rector que guía todo este análisis.

Módulo 8 – Capítulo 04 – Sección 06

Cierre: el hardware correcto depende del modelo elegido, no al revés

Un error frecuente en la planificación de proyectos de LLMs locales es invertir el orden del proceso: comprar o alquilar hardware primero y luego buscar qué modelo cabe en ese presupuesto. El proceso correcto es el inverso: definir el modelo que cumple los requisitos de calidad en la tarea específica, calcular sus necesidades de VRAM con las fórmulas establecidas en este capítulo, y entonces seleccionar el hardware que satisface esas restricciones al menor costo total de propiedad. Un equipo que compra una GPU de 16 GB sin considerar el KV cache para contextos largos descubrirá que su modelo de 13B no cabe en producción cuando los usuarios empiezan a enviar documentos largos; un equipo que sobredimensiona hacia H100 para un workload de inferencia de 7B con baja concurrencia paga 10x más de lo necesario.

El análisis de hardware que este capítulo ha construido —del tipo de arquitectura (GPU, CPU, Apple Silicon) a la VRAM necesaria (pesos + KV cache + overhead), al catálogo de GPUs disponibles, a la ventaja de la memoria unificada de Apple Silicon y al análisis de costo-rendimiento cloud vs local— proporciona todas las piezas para tomar esa decisión de forma sistemática. La madurez del ecosistema de hardware para LLMs en 2025 ofrece opciones viables en cada punto de precio: modelos de 1B-3B ejecutables en CPUs modernas sin GPU, modelos de 7B-13B en GPUs de consumo de 8-16 GB de VRAM, modelos de 13B-34B en Apple Silicon con memoria unificada de 32-64 GB, y modelos de 70B-405B en multi-GPU de datacenter.

La cuantización es la palanca que ajusta el tamaño del modelo al hardware disponible. Si el modelo elegido por sus cualidades de calidad no cabe en el hardware disponible, la cuantización permite reducirlo: de BF16 a Q8 se elimina el 50% del tamaño; de Q8 a Q4_K_M se elimina otro 50%, con una degradación de calidad que para la mayoría de las tareas de producción es inferior al umbral perceptible. Esta combinación de selección de modelo por calidad y ajuste de cuantización por hardware produce sistemas más eficientes que los que resultan de seleccionar el hardware primero y aceptar el modelo que quepa.

Conocer los requisitos de hardware antes de comenzar el diseño de la arquitectura de despliegue no solo evita sorpresas tardías sino que permite mantener una conversación fundamentada con los stakeholders sobre los costos de infraestructura. Presentar a la dirección técnica el análisis de break-even con los números reales de VRAM necesaria, las opciones de hardware viables y el costo mensual proyectado para cada opción es exponencialmente más útil que solicitar "un presupuesto para GPU" sin contexto.

Buena práctica

Mantener una hoja de cálculo de requisitos de hardware actualizada con los modelos candidatos, sus variantes de cuantización, los requisitos de VRAM calculados con las fórmulas de este capítulo y las GPUs que los soportan. Esta tabla de referencia acelera la toma de decisiones en los proyectos y evita el análisis desde cero en cada nuevo proyecto.

"Premature optimization is the root of all evil — but failing to estimate resource requirements before committing to an architecture is the root of most engineering disasters." — Donald Knuth, adaptado al contexto de infraestructura ML, recordando que el análisis de capacidad debe preceder siempre a la selección de hardware.